SALUNIC - Sistema de Salud y Bienestar

Módulo por Consola

DataClass | Cola con Prioridad | Búsqueda Lineal | Archivo de Datos

Guía para Estudio

ÍNDICE

1. Objetivo
2. Estructura del Módulo por Consola
3. DataClass (entidades.py)
4. Cola con Prioridad (cola_atencion.py)
5. Búsqueda Lineal (busqueda_lineal.py)
6. Archivo de Datos JSON (archivo_datos.py)
7. Integración con main.py
8. Pruebas y Ejemplos
9. Conclusión

1. Objetivo

El objetivo de este módulo es aplicar los conceptos de Programación Estructurada: clases con DataClass, estructuras de datos (cola con prioridad), algoritmos de búsqueda (búsqueda lineal) y persistencia de datos (archivos JSON).

2. Estructura del Módulo por Consola

Ubicación: consola/

Archivos principales:
- entidades.py: Definición de las clases con @dataclass
- datos.py: Datos iniciales en memoria
- funciones.py: Funciones CRUD y lógica del sistema
- cola_atencion.py: Cola con prioridad usando heapq
- busqueda_lineal.py: Implementación de búsqueda lineal
- archivo_datos.py: Guardar/cargar datos en JSON
- main.py: Menú principal e integración

3. DataClass (entidades.py)

Los dataclasses permiten crear clases para representar entidades con menos código. Se importan con: from dataclasses import dataclass

Entidades definidas:

Usuario

- id: int
- nombre: str
- email: str
- password: str
- telefono: str = "+505 00000000"

CitaMedica

- id: int
- paciente_id: int
- medico_id: int
- especialidad: str
- fecha: str (DD/MM/AAAA)
- hora: str (HH:MM)
- lugar: str = "Hospital"

Medicamento

- id: int
- nombre: str
- dosis: str
- frecuencia: str
- presentacion: str = "Tabletas"

AlmacenMedicamento

- id: int
- medicamento_id: int
- cantidad: int
- fecha_vencimiento: str (DD/MM/AAAA)

4. Cola con Prioridad (cola_atencion.py)

Se utiliza heapq para implementar una cola con prioridad. El objetivo es atender primero a Adultos Mayores, sin importar el orden de llegada.

Estrategia:

- Se usa una tupla (prioridad, turno_global, item) para ordenar
- Prioridad 0 = Adulto Mayor (máxima prioridad)
- Prioridad 1 = Paciente Normal
- turno_global actúa como desempate para mantener un orden estable
- heapq.heappush() agrega y heapq.heappop() extrae el elemento con menor prioridad numérica (prioritario)

Métodos principales:

- encolar(item, es_adulto_mayor=False): Agrega paciente con su prioridad
- desencolar(): Atiende y remueve el paciente con mayor prioridad
- mostrar_cola(): Muestra la cola ordenada por prioridad y turno
- tamanio(): Retorna cantidad de pacientes en espera
- esta_vacia(): Verifica si hay pacientes en espera

5. Búsqueda Lineal (busqueda_lineal.py)

La búsqueda lineal recorre la lista elemento por elemento, comparando uno a uno hasta encontrar coincidencias.

Función general:

busqueda_lineal(lista, criterio, atributo) - Recorre con un ciclo for, usa getattr() para obtener el atributo, compara por igualdad (case-insensitive para strings), retorna lista de resultados.

Funciones específicas:

- busqueda_lineal_por_email(usuarios, email): Busca un usuario exacto por email
- busqueda_lineal_citas_por_medico(citas, medico_id): Filtra citas por ID de médico
- busqueda_lineal_medicamento_por_nombre(medicamentos, nombre): Filtra medicamentos por nombre

6. Archivo de Datos JSON (archivo_datos.py)

Se utiliza JSON para guardar y cargar la información. Los objetos DataClass se convierten a diccionarios con dataclasses.asdict() y se reconstruyen con Clase(**dict).

Archivo: consola/salunic_datos.json

guardar_datos():

1. Construye un diccionario con las 4 listas (usuarios, citas, medicamentos, almacen)
2. Convierte cada objeto a dict con asdict()
3. Guarda con json.dump() con ensure_ascii=False e indent=4

cargar_datos():

1. Verifica si existe el archivo
2. Lee con json.load()
3. Limpia las listas actuales con clear()
4. Reconstruye objetos DataClass con Usuario(**u), etc.

7. Integración con main.py

El menú del módulo por consola integra todos los componentes:

- Al iniciar, llama a cargar_datos() para recuperar información guardada
- Permite gestionar usuarios, citas, medicamentos
- Tiene opción "Cola de Atención con Prioridad" para encolar/desencolar pacientes
- Tiene opción "Búsqueda Lineal" para buscar por diferentes criterios
- Permite guardar con guardar_datos() antes de salir

8. Pruebas y Ejemplos

Ejemplo: Cola con Prioridad

Si se encolan: Paciente Normal (llega primero) y luego Adulto Mayor, el Adulto Mayor será atendido PRIMERO porque tiene prioridad 0.

Ejemplo: Búsqueda Lineal

Al buscar por email "ana@salunic.com", se recorre la lista elemento por elemento comparando u.email hasta encontrar la coincidencia.

Ejemplo: Persistencia JSON

Al guardar, se convierte todo a JSON. Al cargar, se reconstruyen los objetos DataClass manteniendo sus atributos.

9. Conclusión

El módulo por consola cumple con todos los requisitos solicitados:

- DataClass: Implementado en entidades.py con 4 clases
- Cola con Prioridad: Implementado con heapq, dando prioridad al Adulto Mayor
- Búsqueda Lineal: Implementado recorriendo elemento por elemento
- Archivo de Datos: Implementado con JSON (guardar/cargar)

Todo esto se integra correctamente en main.py y funciona tanto de forma aislada como junto al resto del sistema SALUNIC.